

Implementing Cryptographic Software

Why production libraries need careful details

Lei Xu

lxu12@kent.edu

Computer Science Department
Kent State University

Lecture roadmap

- Implementation contracts
- Bytes and API boundaries
- Heartbleed case study
- Lengths and native interfaces
- Failure handling
- Runtime state and secret handling
- Optimization and testing
- Review and discussion

Implementation contracts

The implementation problem

Assume we already have a specified, reviewed encryption scheme.

- How do we represent its inputs exactly?
- How do we implement it safely on real machines?
- How do callers handle its outputs and failures?
- How do we check that an optimization preserves behavior?

Background: byte strings, loops, and function calls.

No cipher-design knowledge is required.

Where the implementation work lives

Layer

Responsibilities

Application

Input format, key storage, failure handling

Language binding

Types, lengths, buffer ownership, exceptions

Library implementation

Exact arithmetic, memory safety, secret handling

Platform support

Operating-system services and processor-specific code

We will follow a byte string through these layers.

A small public interface

Python with PyNaCl, a binding to libsodium:

```
from nacl.secret import SecretBox
from nacl.utils import random
```

```
key = random(SecretBox.KEY_SIZE)
box = SecretBox(key)
packet = box.encrypt(b"pay=100")
message = box.decrypt(packet)
```

- message is b"pay=100"
- b indicates a byte string
- The library operation is supplied. We will examine the surrounding implementation work

Source: [PyNaCl: SecretBox API](#).

The contract of our supplied operation

- **Key:** a secret byte string of the required size
- **Nonce:** a public per-message value, unique under a key for this API
- **Tag:** verification data that the operation produces and checks
- **Decrypt result:** verified bytes, or an error

Our task is to preserve this contract at every software boundary.

Source: [Libsodium: SecretBox contract](#).

Bytes and API boundaries

Text, bytes, and hexadecimal

Text	UTF-8 bytes in hex	Byte count
A	41	1
41	34 31	2
é	c3 a9	2

- Here, é is the single character U+00E9
- A cipher operates on the encoded bytes
- Hexadecimal lets us display arbitrary bytes as text

Prediction: What goes wrong if we use character count as byte count?

Bug: using character count as byte count

```
text = "café"  
data = text.encode("utf-8")  
count = len(text) # 4 characters  
sent = data[:count] # only 4 of 5 bytes
```

Expected bytes 63 61 66 c3 a9 5 bytes

Sent bytes 63 61 66 c3 4 bytes

- The last character is incomplete, so strict UTF-8 decoding fails
- **Fix:** compute the length after encoding: `len(data)`

Bug: treating binary data as a C string

Buffer 41 00 42 00

Intended message 41 00 42, length 3

C string length `strlen` stops at the first zero, giving 1

- Zero is a valid byte in a key, ciphertext, or binary message
- A pointer identifies a memory location, not the data length
- The interface must carry an explicit byte count
- The final zero in this example terminates the C string safely
- An arbitrary binary buffer may have no terminator at all

Hex parsing needs an explicit policy

Input

Potential policy

41 and 4A

Accept upper- and lowercase digits

4

Reject the incomplete byte

4g

Reject the invalid digit

41 42

Reject the space

- Python's `bytes.fromhex` accepts ASCII whitespace
- Calling a parser alone may or may not enforce the intended policy
- A library can choose a different documented policy

Source: [Python: `bytes.fromhex`](#).

Boundary inputs belong in the interface contract

Input to SecretBox

Expected behavior

Empty plaintext

Accept and produce a packet

Binary plaintext with zeros

Preserve every byte

Key with the wrong size

Reject at object construction

Truncated or altered packet

Reject during decryption

The text decoder runs only after the library returns verified bytes.

Source: [PyNaCl: input and failure contracts](#).

Output size depends on the API

For a 7-byte message	Overhead	Total
C SecretBox output	16-byte tag	23 bytes
PyNaCl complete packet	24-byte nonce + 16-byte tag	47 bytes

- The C call receives the nonce separately
- Its caller must provide enough writable output space
- The Python binding manages output storage for its caller

Using the message length as the output capacity is insufficient.

Source: [Libsodium: C buffer contract](#).

Source: [PyNaCl: packet packaging](#).

Heartbleed case study

Heartbleed case study: the heartbeat request

- A heartbeat checks whether the other endpoint responds
- The receiver echoes the request's payload

Request field	Bytes	Example
Type	1	Heartbeat request
Claimed payload length	2	3
Payload	3	CAT
Padding	16	Extra bytes the receiver ignores

- This example occupies 22 bytes. Its response echoes CAT
- The response has its own header and padding

Source: [RFC 6520: heartbeat messages](#).

Heartbleed: the claimed length exceeds the input

Heartbleed (CVE-2014-0160), disclosed in April 2014, affected OpenSSL's heartbeat implementation.

- Use the preceding slide's request: a 3-byte header, CAT (3 bytes), and 16 padding bytes
- Change only the claimed payload length from 3 to 24

	Normal request	Malformed request
Claimed payload length	3	24
Received record length	22	22

- The vulnerable code copied the claimed number of bytes without checking the received record length
- The real bug could expose roughly 64 KB of process memory per request

Prediction: what could the extra response bytes contain?

Source: [OpenSSL: April 7, 2014 advisory](#).

The copy crosses the received-record boundary

Illustrative memory layout with a fictional adjacent value:

Offsets	Contents	Region
0–2	Type and claimed length	Received record
3–5	CAT	Received record
6–21	16 padding bytes	Received record
22–26	TOKEN	Outside the received record

- Copying 24 bytes from offset 3 returns CAT, the padding, and TOKEN
- **A large destination buffer does not make the source read valid**
- Actual leaked contents depended on the process's memory

Source: [OpenSSL: heartbeat read overrun](#).

The missing check belongs before the copy

Simplified pseudocode for a heartbeat request's length checks:

```
if received < 3 + 16: discard
claimed = read_payload_length(header)
if claimed > received - 3 - 16: discard
copy(response_payload, input + 3, claimed)
```

- Check the minimum size before reading fields or subtracting
- Reserve the 3-byte header and at least 16 bytes of padding
- Discard an invalid request before constructing a response

Assume the request type and protocol state are valid. Response allocation and other protocol checks are omitted here.

Source: [RFC 6520: length and padding rules](#).

Tests that expose the missing validation

Test request

Expected result

3-byte payload, length 3, 16 padding bytes

Echo CAT

Same bytes, but claimed length 24

Discard

Only 2 header bytes

Discard

3-byte payload, length 3, only 15 padding bytes

Discard

3-byte payload, length 3, 17 padding bytes

Echo CAT

For invalid input, verify that no copy or response occurs.

Discussion: why would testing only valid heartbeat requests miss this bug?

Lengths and native interfaces

Bug: a length calculation wraps around

Toy machine: an unsigned length field holds values from 0 to 255.

message length = 250, overhead = 16.

- The required capacity is 266 bytes
- Storing 266 in this 8-bit field leaves the value 10
- An allocator might reserve 10 bytes for a 266-byte output
- **The size calculation must reject overflow**

This toy models a fixed-width length conversion. Python integers do not wrap this way automatically.

Length checks must precede arithmetic

Pseudocode for a bounded output size:

```
require 0 <= overhead <= LIMIT
require 0 <= length <= LIMIT - overhead
capacity = length + overhead
output = allocate(capacity)
if allocation_failed(output):
    return error
```

- LIMIT respects the size type, the API, and the application's limit
- An implementation must validate a narrowing integer conversion too
- Allocating memory and writing into it are separate responsibilities

Byte order changes the value a machine reads

Bytes	Interpretation	Value
01 00	Most significant byte first	256
01 00	Least significant byte first	1

```
raw = bytes.fromhex("0100")
```

```
int.from_bytes(raw, "big") # 256
```

```
int.from_bytes(raw, "little") # 1
```

- Implement the byte order specified by the format or algorithm
- The machine's preferred order does not define the external format

Source: [Python: integer conversion](#).

A language binding must translate the contract

- Convert language objects into the exact native types
- Check lengths before passing a pointer into native code
- Keep backing memory alive while the library uses it
- Translate native failures into the expected exception or result
- Return only the bytes that the operation actually produced

Potential gaps

- A Python integer can exceed the range of a native length type
- A valid address can still point to a buffer that is too short

Failure handling

Success is not always the value 1

Libsodium's `crypto_secretbox_open_easy` returns:

- 0 Verification and decryption succeeded
- 1 Verification failed

For this function, test the documented status explicitly:

```
if (status != 0) {  
    return error;  
}  
use_plaintext();
```

A generic “nonzero means success” assumption reverses this contract.

Source: [Libsodium: decryption return values](#).

Bug: continuing after a decryption error

Python example with a valid packet followed by an invalid one:

```
message = b""
for packet in packets:
    try:
        message = box.decrypt(packet)
    except CryptoError:
        print("Rejected")
    process(message)
```

Prediction: what does process receive after the second call?

Failure handling must block the next operation

```
for packet in packets:
    try:
        message = box.decrypt(packet)
    except CryptoError:
        print("Rejected")
        continue
    process(message)
```

- In the buggy version, the failed assignment leaves the old value in `message`. The application processes it again
- In this version, failure skips processing for that packet

The demonstration tests the failure path

Case

Valid packet

Truncated packet

Packet with one changed byte

Decryption under another key

Valid empty plaintext

Observable result

Process the recovered bytes once

Report failure and process nothing

Report failure and process nothing

Report failure and process nothing

Process a valid empty byte string

- An exception message alone does not show that processing stopped
- The demo also records the calls to process

Runtime state and secret handling

Random generation depends on system services

- The library obtains unpredictable bytes through operating-system services
- Native callers initialize libsodium with `sodium_init`
- Initialization can fail, and the caller must handle that result
- An application must not substitute a timestamp when secure generation fails

The Python binding handles native initialization.

The implementation still needs a defined failure path underneath the short call.

Source: [Libsodium: initialization](#).

Source: [Libsodium: random generation](#).

Counter state must survive concurrency and restarts

Suppose an application supplies counter-based nonces under one key.

Step	Worker A	Worker B
Read shared counter	Reads 12	Reads 12
Encrypt	Uses 12	Uses 12
Store next counter	Writes 13	Writes 13

Both workers used the same value. Separate reads and writes are insufficient.

- Counter allocation needs coordination
- Restart and restore must not roll the counter back under the same key

Our PyNaCl example delegates nonce generation to the binding.

Source: [PyNaCl: nonce requirements](#).

Secrets also have a lifetime in memory

- An application can copy a key into logs or temporary buffers
- An ordinary memory-clearing operation may disappear during optimization
- Libsodium offers `sodium_memzero` for explicit clearing
- Clearing one buffer does not erase other copies

In Python, deleting a variable does not establish that every copy of its bytes has been erased.

The application must consider where secrets travel and how long they remain.

Source: [Libsodium: secure memory](#).

A comparison can reveal how far it matched

Toy comparison of equal-length byte strings:

```
def early_equal(a, b):  
    for i in range(len(a)):  
        if a[i] != b[i]:  
            return False  
    return True
```

- The result says whether the strings match
- The amount of work also depends on the first mismatch
- We will count comparisons to illustrate a possible timing leak

Equal answers, different amounts of work

Stored toy secret: 7392

Guess	Result	Comparisons
0000	False	1
7000	False	2
7300	False	3
7390	False	4

- A longer matching prefix causes more comparisons here
- This demonstration counts operations. Real timing measurements include noise and require additional analysis

Production comparison has a stronger contract

Example C call for two equal-length byte buffers:

```
int same = (sodium_memcmp(a, b, n) == 0);
```

- Comparison time is independent of byte contents for fixed length
- This use of “constant time” differs from $O(1)$ complexity
- Source code alone can hide compiler and processor effects
- A homemade Python loop gives no constant-time guarantee

Use the library’s verification operation when checking its tags.

Source: [Libsodium: constant-time comparison](#).

Optimization and testing

One algorithm can have several implementations

- A portable implementation supports a broad set of processors
- An optimized implementation may process several words at once
- Runtime selection chooses code supported by the current processor
- Every implementation must produce the same specified result

A faster path also needs correct handling of short inputs and leftover bytes.

Compiler transformations and secret-dependent memory access need review too.

Source: [Libsodium: runtime selection and implementation concerns](#).

Optimization creates boundary cases

Suppose an optimized loop processes 16 bytes per iteration.

Input length	What it exercises
---------------------	--------------------------

0	No iterations
---	---------------

1 and 15	Only leftover bytes
----------	---------------------

16	Exactly one full group
----	------------------------

17	One full group and one leftover byte
----	--------------------------------------

31, 32, and 33	The next group boundary
----------------	-------------------------

- Compare optimized results against an independent reference
- Check memory accesses as well as output bytes

Different checks expose different implementation bugs

Check

Known expected output

Round trip

Failure-path checks

Cross-implementation tests

Memory-analysis tools

What it can expose

Wrong byte order or a mistaken algorithm step

Lost or truncated data on tested inputs

Use of stale or unverified output

Differences between portable and fast paths

Out-of-bounds reads, writes, or invalid lifetimes

Two matching mistakes in encryption and decryption can pass a round trip.

Source: [Libsodium: testing and analysis](#).

Malformed inputs exercise the surrounding code

- Try truncations at different byte positions
- Vary length fields independently of the actual buffer
- Exercise empty, oversized, and malformed encoded inputs
- Use fuzzing to generate many variations automatically

Expected behavior: bounded resource use, controlled errors, and no processing of rejected input.

Fuzzing finds bugs through executions. It does not prove their absence.

Maintenance continues after the first correct version

- A compiler change can alter the generated machine code
- A new processor may select a different implementation
- A binding update can change how errors reach callers
- A dependency update needs regression checks in the application
- Record the versions used to reproduce a failure
- Keep minimized failure cases as regression tests

Review and discussion

Discussion: what fails in this implementation?

Name the consequence and the fix for each case.

1. A native call receives character count as its byte count
2. A length calculation wraps before an output allocation
3. The program logs a decryption error and uses the previous result
4. A command prints uppercase hex instead of required lowercase hex

Which failures affect memory safety or accepted data?

Which concern an agreed output convention?

Discussion answers

1. **Byte count:** part of the encoded input may disappear. Encode first, then measure the bytes
2. **Overflow:** the output buffer may be too short. Check bounds before arithmetic and conversion
3. **Error handling:** stale data may be processed again. Stop or skip that input after a failed operation
4. **Hex case:** the command violates the required format. Produce the representation promised by its interface

Every requirement needs a reason. The consequences differ in severity.

Follow-up question

The encryption algorithm is fixed and reviewed.

How can its implementation still fail?

- Give one bug inside a library or binding and one bug in its caller
- For each, propose a check that would expose it

References and further exploration

- [OpenSSL: Heartbleed advisory, April 2014](#)
- [RFC 6520: heartbeat format and validation](#)
- [Libsodium: buffer sizes and return values](#)
- [PyNaCl: Python inputs, outputs, and exceptions](#)
- [Libsodium: comparison and encoding helpers](#)
- [Libsodium: secure memory](#)
- [Libsodium: implementations, platforms, and testing](#)
- [Python: bytes and integer conversions](#)

The snippets illustrate implementation contracts and bugs.

They do not implement or redesign a cipher.